

南開大學

《汇编语言与逆向技术》课程实验报告

实验八：Reverse Engineering Exercises



学 院____网络空间安全学院____
专 业____物联网工程____
学 号____2310422____
姓 名____谢小珂____
班 级____0410____

一、实验目的

- 1、熟悉静态反汇编工具 Binary Ninja;
- 2、熟悉反汇编代码的逆向分析过程;
- 3、掌握反汇编语言中的数学计算、数据结构、条件判断、分支结构的识别和逆向分析

二、实验原理

1. 软件逆向工程：在不具备源代码的情况下，通过对可执行文件中的指令和数据进行分析，恢复程序结构和功能，应用于漏洞挖掘、恶意代码分析及软件保护等安全领域。
2. 静态逆向分析：在不实际运行程序的前提下，通过反汇编代码、控制流图等静态方式分析程序行为。该方法安全性高，但对汇编语言和程序结构理解要求较高。
3. 实验环境：分析对象为 Windows PE 文件 task1/2.exe，使用 Binary Ninja 进行静态分析。该工具支持反汇编 x86/x64 代码，自动识别函数、基本块和字符串，提供线性反汇编视图与控制流图（CFG），并可通过交叉引用跟踪函数调用与跳转关系。

三、实验过程

3.1 二进制代码的反汇编代码及图形化视窗

使用 Binary Ninja 打开待分析的 PE 文件，在视图选项中切到线性视图（Linear / Disassembly），让每条指令顺序显示，得到二进制代码的反汇编代码，如图 1 图 2 所示。

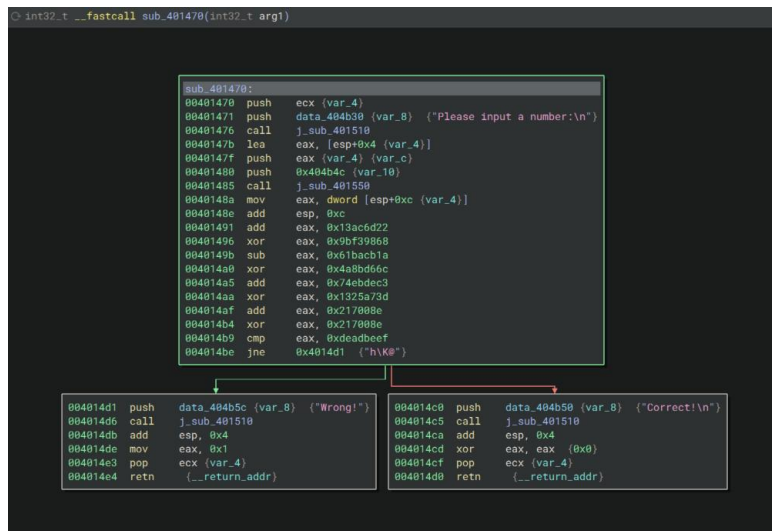


图 4 task2.exe 的反汇编代码的图形化显示

3.2 逆向分析

对反汇编代码和计算过程、条件判断、分支结构等信息进行分析，逆向推出程序的正确输入数据，完成逆向分析挑战。

(一) task1

1) 0x401470 开头一提示+读入输入

建立栈帧后 push "Please input a string" 调用打印函数；随后把 var_54（输入缓冲区）地址传给读入函数，把字符串读到栈上。

```
sub_401470:
00401470 sub     esp, 0x54
00401473 mov     eax, dword [security_cookie]
00401478 xor     eax, esp {var_54}
0040147a mov     dword [esp+0x50 {var_4}], eax
0040147e push    data_404b30 {__saved_esi} {"Please input a string:\n"}
00401483 call   j_sub_401610
00401488 lea     eax, [esp+0x4 {var_54}]
0040148c push    eax {var_54} {var_5c}
0040148d push    0x404b4c {var_60}
00401492 call   j_sub_401650
00401497 lea     ecx, [esp+0xc {var_54}]
0040149b add     esp, 0xc
0040149e lea     edx, [ecx+0x1 {var_53}]
```

2) 0x4014a1—strlen 扫描循环

mov al,[ecx] → inc ecx → test al,al → jne: 逐字节找 '\0'，典型 strlen 逻辑。

```
004014a1 mov     al, byte [ecx]
004014a3 inc     ecx
004014a4 test    al, al
004014a6 jne     0x4014a1
```

3) 0x4014a8—长度必须为 8

sub ecx,edx ; cmp ecx,0x8 ; je 0x4014bc: 若长度不等于 8, 不跳转继续(进入错误分支)。

```
004014a8 sub    ecx, edx {var_53}
004014aa cmp    ecx, 0x8
004014ad je     0x4014bc
```

4) 0x4014af—长度错误输出

push "Wrong length\n" ; call: 长度不为 8 时直接打印并结束。

```
004014af push    data_404b50 {__saved_e si} {"Wrong length\n"}
004014b4 call    j_sub_401610
004014b9 add     esp, 0x4
```

5) 0x4014bc~0x4014ce—约束①: 第 1+第 2 字符

movsx 取第 1、2 字符(有符号扩展), lea eax,[edx+ecx] ; cmp eax,0xb2:
input[0]+input[1]=0xb2(178)

```
004014bc movsx   ecx, byte [esp {var_54}]
004014c0 movsx   edx, byte [esp+0x1 {var_53}]
004014c5 push    esi {__saved_e si}
004014c6 lea     eax, [edx+ecx]
004014c9 cmp     eax, 0xb2
004014ce jne     0x401596 {"h`K@"}
```

6) 0x4014d4~0x4014d9—约束②: 第 1-第 2 字符

sub ecx,edx ; cmp ecx,0xFFFFFDDA(-38): input[0]-input[1]=-38
联立解得: input[0]='F', input[1]='l'。

```
004014d4 sub     ecx, edx
004014d6 cmp     ecx, 0xfffffdda
004014d9 jne     0x401596 {"h`K@"}
```

7) 0x4014df~0x4014f4—约束③: 第 4、第 6 字符(LEA 做乘法)

$3*input[3] + 2*input[5] = 0x21D(541)$ 。

```
004014df movsx   edx, byte [esp+0x7 {var_51}]
004014e4 movsx   ecx, byte [esp+0x9 {var_4f}]
004014e9 lea     eax, [edx+edx*2]
004014ec lea     eax, [eax+ecx*2]
004014ef cmp     eax, 0x21d
004014f4 jne     0x401596 {"h`K@"}
```

8) 0x4014fa~0x401509—约束④: 第 4、第 6 字符

$input[5] - 8*input[3] = 0xFFFFFD3C(-708)$

联立解得: input[3]='g', input[5]='t'。

```

004014fa lea     eax, [edx*8]
00401501 sub     ecx, eax
00401503 cmp     ecx, 0xfffffd3c
00401509 jne     0x401596 {"h`K@"}

```

9) 0x40150f~0x401527—约束⑤：第 5、第 8 字符

imul ecx, input[4], 0x7D + lea 组合：

$$125 \times \text{input}[4] + 18 \times \text{input}[7] = 0x2AD9 (10969)$$

```

0040150f movsx   esi, byte [esp+0x8 {var_50}]
00401514 movsx   edx, byte [esp+0xb {var_4d}]
00401519 imul    ecx, esi, 0x7d
0040151c lea     eax, [edx+edx*8]
0040151f lea     eax, [ecx+eax*2]
00401522 cmp     eax, 0x2ad9
00401527 jne     0x401596 {"h`K@"}

```

10) 0x401529~0x40153d—约束⑥：第 5、第 8 字符

shl eax, 5 ; add eax, esi 得到 $33 \times \text{input}[4]$ ；lea 得到 $6 \times \text{input}[7]$ ：

$$6 \times \text{input}[7] - 33 \times \text{input}[4] = 0xFFFFF613 (-2541)$$

联立解得：input[4]='S'，input[7]='!'。

```

00401529 mov     eax, esi
0040152b lea     ecx, [edx+edx*2]
0040152e shl     eax, 0x5
00401531 add     ecx, ecx
00401533 add     eax, esi
00401535 sub     ecx, eax
00401537 cmp     ecx, 0xfffff613
0040153d jne     0x401596 {"h`K@"}

```

11) 0x40153f~0x401559—约束⑦：第 3、第 7 字符

lea 得到 $10 \times \text{input}[6]$ 与 $3 \times \text{input}[2]$ ：

$$10 \times \text{input}[6] - 3 \times \text{input}[2] = 0x351$$

```

0040153f movsx   edx, byte [esp+0xa {var_4e}]
00401544 movsx   esi, byte [esp+0x6 {var_52}]
00401549 lea     ecx, [edx+edx*4]
0040154c add     ecx, ecx
0040154e lea     eax, [esi+esi*2]
00401551 sub     ecx, eax
00401553 cmp     ecx, 0x351
00401559 jne     0x401596 {"h`K@"}

```

12) 0x40155b~0x401575—约束⑧：第 3、第 7 字符

shl/sub/add 组合出 $62 \times \text{input}[2]$ ；lea eax, [edx*8]；sub eax, edx 出 $7 \times \text{input}[6]$ ：

$62 \times \text{input}[2] - 7 \times \text{input}[6] = 0x1460 (5216)$

联立解得: $\text{input}[2] = 'a'$, $\text{input}[6] = 'r'$ 。

```
0040155b  mov     ecx, esi
0040155d  lea     eax, [edx*8]
00401564  shl     ecx, 0x5
00401567  sub     eax, edx
00401569  sub     ecx, esi
0040156b  add     ecx, ecx
0040156d  sub     ecx, eax
0040156f  cmp     ecx, 0x1460
00401575  jne     0x401596 {"h`K@"}
```

13) 0x401577~0x401595—全部通过输出 Correct

所有 cmp 都满足且未跳到失败地址, 则 push "Correct\n"; call, 返回 0。

```
00401577  push    data_404b68 {"Correct\n"}
0040157c  call    j_sub_401610
00401581  add     esp, 0x4
00401584  xor     eax, eax {0x0}
00401586  pop     esi {__saved_esi}
00401587  mov     ecx, dword [esp+0x50 {var_4}]
0040158b  xor     ecx, esp {var_54}
0040158d  call    CookieCheckFunction
00401592  add     esp, 0x54
00401595  retn    {__return_addr}
```

14) 0x401596~0x4015b7—任一失败输出 Wrong

任一条件不满足触发 jne 0x401596, 打印 "Wrong\n", 返回 1。

```
00401596  push    data_404b60 {"Wrong\n"}
0040159b  call    j_sub_401610
004015a0  mov     ecx, dword [esp+0x58 {var_4}]
004015a4  add     esp, 0x4
004015a7  mov     eax, 0x1
004015ac  pop     esi {__saved_esi}
004015ad  xor     ecx, esp {var_54}
004015af  call    CookieCheckFunction
004015b4  add     esp, 0x54
004015b7  retn    {__return_addr}
```

最终正确输入 (长度 8): FlagStr!

(二)task2

1) 0x401470~0x4014be, 主逻辑—读入整数 + 常量链校验

push "Please input a number:\n"; call sub_401510: 打印提示。

lea eax, [var_4]; push eax; push 0x404b4c; call sub_401550: 把 var_4 地址作为输出参数传入读入函数, 按十进制格式读取一个整数到 var_4。

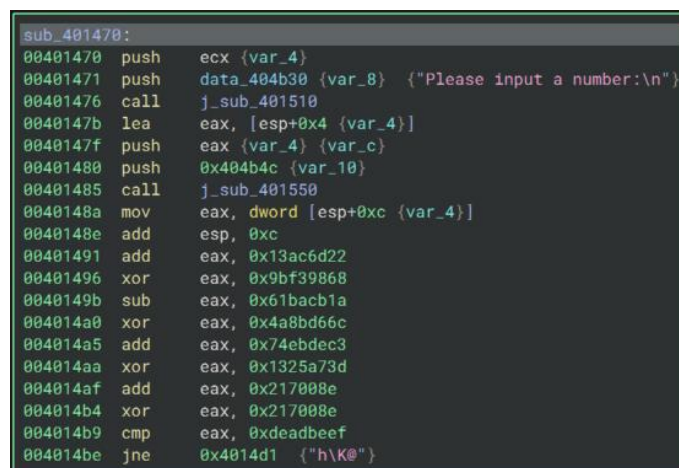
mov eax, [var_4] 后对 eax 做固定的 32-bit 变换（在 $\text{mod } 2^{32}$ 环上执行，溢出自动截断），最后比较目标值：

```
eax = x
eax = eax + 0x13ac6d22
eax = eax ^ 0x9bf39868
eax = eax - 0x61bacb1a
eax = eax ^ 0x4a8bd66c
eax = eax + 0x74ebdec3
eax = eax ^ 0x1325a73d
eax = eax + 0x0217008e
eax = eax ^ 0x0217008e
cmp eax, 0xDEADBEEF
jne 0x4014d1 ; 不等→Wrong
```

由于 add/sub/xor 在 2^{32} 上可逆，可从 0xDEADBEEF 反推输入，得到：
内存里的 32 位输入值应该是 0xD092BFE7。

即无符号十进制：3499278311

有符号十进制：-795688985



```
sub_401470:
00401470 push    ecx {var_4}
00401471 push    data_404b30 {var_8} {"Please input a number:\n"}
00401476 call    j_sub_401510
0040147b lea     eax, [esp+0x4 {var_4}]
0040147f push    eax {var_4} {var_c}
00401480 push    0x404b4c {var_10}
00401485 call    j_sub_401550
0040148a mov     eax, dword [esp+0xc {var_4}]
0040148e add     esp, 0xc
00401491 add     eax, 0x13ac6d22
00401496 xor     eax, 0x9bf39868
0040149b sub     eax, 0x61bacb1a
004014a0 xor     eax, 0x4a8bd66c
004014a5 add     eax, 0x74ebdec3
004014aa xor     eax, 0x1325a73d
004014af add     eax, 0x217008e
004014b4 xor     eax, 0x217008e
004014b9 cmp     eax, 0xdeadbeef
004014be jne     0x4014d1 {"h\K@"}
```

2) 0x4014d1~0x4014e4, Wrong 分支—失败处理

任一条件不满足会 jne 0x4014d1 跳入该块：

push "Wrong!" ; call sub_401510 输出失败提示；mov eax, 1 作为返回值
结束。


```

004014d1 push    data_404b5c {var_8} {"Wrong!"}
004014d6 call    j_sub_401510
004014db add     esp, 0x4
004014de mov     eax, 0x1
004014e3 pop     ecx {var_4}
004014e4 retn     {__return_addr}

```

3) 0x4014c0~0x4014d0, Correct 分支—成功处理

当 `cmp eax, 0xDEADBEEF` 相等时不跳转，顺序进入该块：

`push "Correct!\n"` ; `call sub_401510` 输出成功提示；`xor eax, eax` 置返回
 值 0 结束。

```

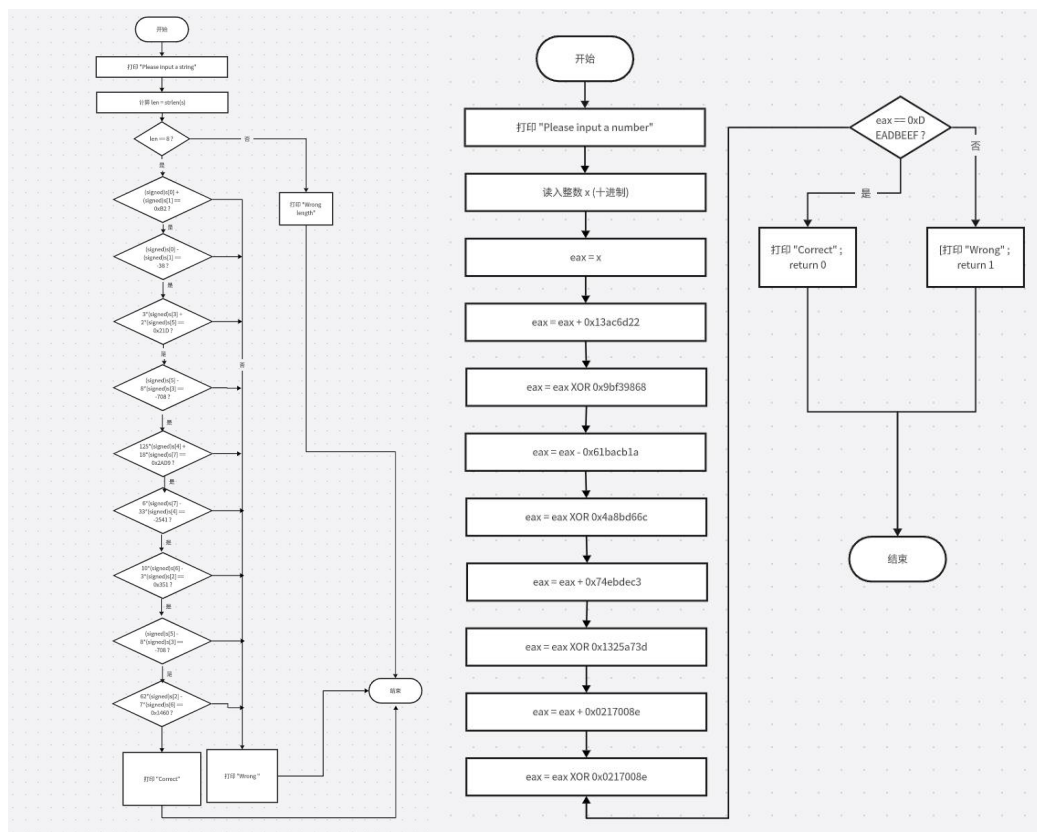
004014c0 push    data_404b50 {var_8} {"Correct!\n"}
004014c5 call    j_sub_401510
004014ca add     esp, 0x4
004014cd xor     eax, eax {0x0}
004014cf pop     ecx {var_4}
004014d0 retn     {__return_addr}

```

最终可用输入（按十进制输入）： 3499278311 或 -795688985

3.3 流程图

左边为 task1, 右边为 task2



3.4 运行结果截图

(一)task1

```
PS K:\大三上\汇编实验\实验报告\实验八\Lab8-REExercises(simple)> ./task1.exe
Please input a string:
FlagStr!
Correct
```

(二)task2

```
PS K:\大三上\汇编实验\实验报告\实验八\Lab8-REExercises(simple)> ./task2.exe
Please input a number:
3499278311
Correct!
PS K:\大三上\汇编实验\实验报告\实验八\Lab8-REExercises(simple)> ./task2.exe
Please input a number:
-795688985
Correct!
```

四、实验结论及心得体会

本次实验让我更熟悉用 Binary Ninja 从“输入点—关键比较点—正确/错误分支”快速定位校验逻辑。task1 里通过 test/jne 识别 strlen, 结合 movsx (有符号扩展) 与 lea (实现乘法/线性组合) 把汇编还原为约束方程并求解。task2 里利用 32 位模 (2^{32}) 运算特性, 基于 add/sub/xor 的可逆性从 0xDEADBEEF 反推输入, 同时也验证了读入函数只按十进制解析导致 0x... 形式输入失败。